vesuvius-challenge-surface-detection

10th place solution

Rank	10
Team	Vibes & Scrolls Trade-off
Author	Tom
Topic ID	679227
Version	Original

Source: https://www.kaggle.com/competitions/vesuvius-challenge-surface-detection/discussion/679227
Retrieved with Kaggle CLI on 2026-07-07.

We thank Kaggle and the Vesuvius Challenge organizers for a fascinating competition.

Although majority of training pipeline was implemented outside the original nnUNet framework, most architectures are derived from work produced by MIC-DKFZ. We are grateful to them for advancing the 3D segmentation domain.

Below is an overview of our approach.

1st Stage — Initial Segmentation

We trained three independent models:

- **ResEnc-L UNet** (4 × TTA): The standard nnUNet-style residual encoder UNet with channels (32, 64, 128, 256, 320, 320) and `n_blocks=(1,3,4,6,6,6)`.
- **Primus-B**: A transformer based segmentation model from dynamic-network-architectures library of MIC-DKFZ.
- **Primus-B V2**: An improved variant of Primus-B.

Models were first pretrained on approximately 1,600 annotated images from other scrolls: https://dl.ash2txt.org/datasets/seg-derived-recto-surfaces/. Labels were slightly different but helped fine-tuning afterward to converge much faster. For example, Primus was taking 700 epochs or more to fully converge, and with pretraining it took 400.

All models were trained at patch size 160^3, with AdamW (`lr=5e-5`, `wd=1e-4`) and mixed-precision training for approximately 400 epochs.

For losses, we used a combination of Dice Loss, Cross-Entropy Loss, Skeleton Recall Loss, and Surface Dice Loss (a modified version of clDice for 2D manifolds in 3D environments; implementation was for CryoET membranes). (Note @sersasj: I had the opportunity to watch Lorenz Lamm's presentation at the CZII competition workshop, very nice to find a use almost a year later in a totally different task.)

From local evaluation, the best models were Primus-B V2, ResEnc-L and Primus-B respectively. We will add scores later. The good thing is that the models are very complementary to each other.

2nd Stage — Ensemble

The idea here was to let a model learn the complementarity between predictions. The model used a 4-channel input: [Image, ResEnc-L binary mask, Primus binary mask, PrimusV2 binary mask].

- **ResEnc-L UNet** ensembles initial stage predictions.

We trained with a random threshold on the fly ranging from 0.1 to 0.7, but used a threshold of 0.3 at inference.

3rd Stage — Refinement

(Note @sersasj: I really liked the discussions about algorithms to fill holes, make the sheets more continuous, and so on. But I'm not clever enough to do this with an algorithm. So I did what I could: train a model to do that.)

We used the same random threshold strategy on the fly. If the threshold is low, we hope the model learns to correct merging sheets, if it is high, we hope the model learns to correct broken sheets. This behavior can be seen in the image below:

We also added a strong `randCoarseDropout` to mask the input to help model learn continuity and fix holes.

Input is [Mask, 2nd stage binary].

- ResEnc-L UNet refinement.

4th Stage — Refinement

Just another refinement network. Same idea of stage 3, help model learn to correct silly errors,

Input is [Mask, 3rd stage binary].

- ResEnc-L UNet — refinement.

5th Stage — Diffeomorphic Stage (biggest boost)

The diffeomorphic network takes the role of shape calibratin and thickness modification. This approach refers Tom's paper (**Perceptual Contrastive Generative Adversarial Network based on image warping for unsupervised image-to-image translation**) and the **FlowNet2**. Unlike classic way like previous stage which just reproduces a better segmentation prediction, this network predicts the stationary velocity field which can manipulate the input mask previous stage it recieves. Intuitively, it is a vector field that determine how the mesh changes in 3d space, and we decide how many step it should move.

Core Designs and Intuition

This model is mainly based on Tom's paper. It performs warping first and then refines the content afterward.

Diffeomorphic Step:

Using a lower threshold to generate the hard mask and applying uniform blur are two crucial steps for helping the model learn the SVF logits. Since the OOF mask is already strong, you need to intentionally introduce "errors" so the model can learn from them and be rewarded for correcting them.

- Using a small threshold reveals more suppressed predictions. Some of these predictions contribute to thicker mask regions, which allows the model to identify and fix such issues.
- A hard mask alone cannot provide useful gradients to the model. We also found that warping a blurred mask is very beneficial for improving topology and boosting VOI performance.

Blurring Example

```
# ----------------------------------------
# 1. Blur the input mask
# ----------------------------------------
Hard_Mask = Soft_Mask>0.3

Blurred_mask = Gaussian_Blur(Hard_Mask, kernel_size=3, sigma=5)
# (Produces soft mask M in [0,1])
```

```
# ---------------------------------------
# 2. Predict stationary velocity field
# ---------------------------------------

gamma = Diffeomorphic_Prediction(Blurred_mask)
# gamma ∈ R^{3×D×H×W}  (SVF logits)

v = tanh(gamma) * max_v
# v : Ω → R³
# bounded stationary velocity field

# ---------------------------------------
# 3. Scaling & Squaring (Lie exponential)
# ---------------------------------------

# Initialize small deformation
phi_0 = v / (2^N)

phi = phi_0

Repeat N times:

    # Compose deformation with itself
    # (φ ∘ φ)(x) = φ(x + φ(x))

    phi = phi + Warp(phi, phi)

# After N steps:
# phi ≈ exp(v)

# ---------------------------------------
# 4. Warp mask with diffeomorphic transform
# ---------------------------------------

Warped_mask(x) = Blurred_mask(x + phi(x))
```

Signed Distance Topology Shift Prediction:

Optical flow–based methods usually suffer from a “folding” issue. This occurs when the model makes abrupt deformations, which significantly damage the topology. Tom struggled with this problem for quite some time. He eventually addressed it by introducing an additional output channel that predicts the “shift” in the SDF space of the warped mask.

```
# ---------------------------------------
# Topology-aware correction
# ---------------------------------------

# Convert to soft signed distance
SDF = log(Warped_mask + ε) - log(1 - Warped_mask + ε)

# Learn correction field from 4th channel r_t
t      = sigmoid(r_t)                # gating map
delta  = max_offset * tanh(r_t)      # signed bounded offset

SDF_corrected = SDF + t * delta

Final_mask = sigmoid(SDF_corrected)
```

Loss functions for warping

- Minimizing SVF Smoothing to avoid folding by surpressing the magnitutes of velocities

```
def svf_smoothness(self, v):
    dz = (v[:, :, 1:] - v[:, :, :-1]).pow(2).mean()
    dy = (v[:, :, :, 1:] - v[:, :, :, :-1]).pow(2).pow(1).mean()
    dx = (v[:, :, :, :, 1:] - v[:, :, :, :, :-1]).pow(2).mean()

    return (dz + dy + dx) / 3.0
```

- Minimizing jacobian log barrier to keep flow active and preventing a compressible elastic material that strongly resists collapsing or flipping

```
def jacobian_determinant(flow):
    """
    flow: (B, 3, D, H, W) displacement field u(x)
    returns: (B, D, H, W) jacobian determinant of φ(x)=x+u(x)
    """
    B, C, D, H, W = flow.shape
    assert C == 3

    # gradients wrt spatial axes (z = depth, y = height, x = width)
    du_dx = torch.gradient(flow, dim=4)[0]  # width axis
    du_dy = torch.gradient(flow, dim=3)[0]  # height axis
    du_dz = torch.gradient(flow, dim=2)[0]  # depth axis

    # components
    ux_x = du_dx[:,0]; ux_y = du_dy[:,0]; ux_z = du_dz[:,0]
    uy_x = du_dx[:,1]; uy_y = du_dy[:,1]; uy_z = du_dz[:,1]
    uz_x = du_dx[:,2]; uz_y = du_dy[:,2]; uz_z = du_dz[:,2]

    # deformation gradient J = I + ∇u
    j11 = 1 + ux_x; j12 =     ux_y; j13 =     ux_z
    j21 =     uy_x; j22 = 1 + uy_y; j23 =     uy_z
    j31 =     uz_x; j32 =     uz_y; j33 = 1 + uz_z

    det = (
        j11 * (j22 * j33 - j23 * j32)
        - j12 * (j21 * j33 - j23 * j31)
        + j13 * (j21 * j32 - j22 * j31)
    )
    return det

def jacobian_log_barrier(flow, eps=1e-6):
    det = jacobian_determinant(flow)
    det_clamped = torch.clamp(det, min=eps)
    loss = -torch.log(det_clamped).mean()
    return loss
```

Loss function for SDF topology fixing

The challenge @Tom faces is avoiding the forth channel cheating and dominating the prediction, which letting the diffeomorphic step useless. We use three loss functions to solve this issue.

- Sparisty loss: Keep sparse topology correction, discouraged from editing everywhere and only activates correction when necessary

```
#suppose t = gating map in topoFix
def topo_sparsity(self, t):
        return t.mean()
```

- Total variation loss: Making topology edits to be region-based, not noisy voxel-wise toggles

```
def topo_tv(self, t):
    dz = (t[:, :, 1:] - t[:, :, :-1]).abs().mean()
    dy = (t[:, :, :, 1:] - t[:, :, :, :-1]).abs().mean()
    dx = (t[:, :, :, :, 1:] - t[:, :, :, :, :-1]).abs().mean()

    return (dz + dy + dx) / 3.0
```

• Boundary loss: Encourage edits near zero-level set and only modify topology near the object surface

```
def topo_boundary(self, t, sdf):
    boundary = torch.exp(-sdf.abs())
    return (t * (1.0 - boundary)).mean()
```

Variations of Implementations

We have two implementation of Diffeomorphic Network, one is in @Tom repositery and another is @Sergio repositery, they have different augmentation, processing and loss functions. All the network architecture are all identical, using the nnUNet-style residual encoder UNet but outputs with 4 channels.

• Loss differneces

Version	CLDice	SoftSDFLoss	Skeleton Recall	Dice + CE	Surface Dice Loss
Tom	o	o	o	o	x
Sergio	x	x	o	o	o

• Augmentation differences

Version	Online thresholding	Mask augmentation
Tom	`threshold = 0.3 + torch.randn(1, device=prob_mask_oof.device) * 0.01` `threshold = torch.clamp(threshold, 0.1, 0.5)`	• Random Affine (3° rotation) • RandCoarseDropout (12 holes, spatial size = 10)

Sergio	`threshold = 0.3 + np.random.uniform(-0.2, 0.5)`	`RandCoarseDropoutdWithRanges(keys=oof_keys, prob=1, shared_holes_range=(10,15), independent_holes_range=(15,15), spatial_size_range=(10,30), fill_value=0.0)`
		
		`RandCoarseDropoutdWithRanges(keys=oof_keys, prob=0.8, shared_holes_range=(20,40), independent_holes_range=(30,60), spatial_size_range=(1,3), fill_value=0.0)`

• Prediciton results: Sergio's vs Tom's

• Learned components: Sergio's vs Tom's

- SVF results: Sergio's vs Tom's

Comparing different diffeomorphic setup

Before the deadline, we conducted simulation trials to approximate the public subset (40 samples per subset, 200 trials) and compared the diffeomorphic networks from the repo of @Tom and @Sergio.

As shown in the plot, there is a clear difference in metric preference between the two approaches. @Tom's model outperforms @Sergio's on Surface Dice, while @Sergio's setup achieves better results on Topology and VOI scores. In most cases, @Sergio's configuration achieves the best overall performance.

However, we also observed that in a few subsets, @Tom's overall score is higher, which is consistent with its LB. This suggests that the public subset may resemble those specific subsets where @Tom's model performs better, and a small number of samples could have disproportionately boosted the public score, essentially a favorable sampling effect.

- Validation results

	CV	LB	PB
Tom	0.619	0.595	0.612
Tom + Sergio	0.621	0.588	0.615

- Hypothesis testing (H0: @Tom = @Sergio, H1: @Tom > @Sergio)

Metric	Mean Difference	T-Statistic	P-Value	Reject?
Surface_dice	+0.0086	74.97	<0.0001	Y
Score	-0.0017	-7.64	1	N
Topo	-0.0124	-20.11	1	N
Voi	-0.004	-117.68	1	N

- Based on the hypothesis testing, these two methods may not have a significant difference in topology or VOI score on the public subset. In that case, the method with the higher surface Dice score dominates, resulting in a 0.007 boost on the leaderboard. The other one we hoped improving lb experiences a substantial drop, scoring 0.588, which does bad in surface dice.

Post-Processing

Post Processing	LB	PB	Comments
remove cc < 3000	0.592	0.614	
x7 median filter	0.596	0.623	
x6 median filter	0.597	0.623	
x8 median filter	0.596	0.624	

Post Processing	LB	PB	Comments
x9 median filter	0.596	0.624	
x10 median filter	0.597	0.624	
binary closing	0.588	0.615	
closing(7)+hole patching+cavity fill	0.583	0.604	
Gaussian smooth + rethreshold + close/open + fill	0.585	0.603	
Remove internal cavities	0.589	0.614	
Erosion / Dilation	0.592	0.613	
Enforced minimum sheet thickness	0.463	0.454	Topo destroyed; Fails small sample
Z-consistent smoothing	0.592	0.615	

Code

https://github.com/Sersasj/vesuvius-challenge-10th-solution

References

- Primus: Enforcing Attention Usage for 3D Medical Image Segmentation — https://openreview.net/forum?id=YWwGmmObri
- MemBrain v2: An end-to-end tool for the analysis of membranes in cryo-electron tomography (Surface Dice Loss) — https://www.biorxiv.org/content/10.1101/2024.01.05.574336v1.full.pdf
- Skeleton Recall Loss for Connectivity Conserving and Resource Efficient Segmentation of Thin Tubular Structures — https://www.ecva.net/papers/eccv_2024/papers_ECCV/papers/09904.pdf
- dynamic-network-architectures library — https://github.com/MIC-DKFZ/dynamic-network-architectures
- PrimusV2 code (for some reason it's not merged, we got lucky to find this fork) — https://github.com/TaWald/dynamic-network-architectures/blob/main/dynamic_network_architectures/architectures/primus.py
- **Perceptual Contrastive Generative Adversarial Network based on image warping for unsupervised image-to-image translation(**https://www.sciencedirect.com/science/article/abs/pii/S0893608023003684**)**
- **FlowNet 2.0: Evolution of Optical Flow Estimation with Deep Networks(**https://arxiv.org/abs/1612.01925**)**

Supplemental Q&A

> **huoxu · 2026-02-28T03:24:56.073000**

I had assumed your solution would incorporate unlabeled data, but it appears not to, which is somewhat unexpected. It appears that semi-supervised methods are not ultimately required for the competition. Having spent a month writing semi-supervised training code, the final results still fell short of those achieved through supervised training.

Sergio Alvarez · 2026-02-28T03:33:35.510000

We did actually experiment with semi-supervised methods. We tried Cross Pseudo Supervision, which seemed to help, but it doubled training time so we eventually dropped it. We also did pseudo-label iterations: training on labeled data, running inference on unlabeled data, pretraining on pseudo-labels, and fine-tuning again, repeating this twice. It helped with convergence in the early stages. However, @p4rallax found additional labeled so we used that to pretrain

Chris Deotte · 2026-03-03T02:40:52.030000

Wow, impressive. Post process makes a big effect in this comp. It's interesting how it affects private LB more than public LB. Does it also improve CV as much as private LB? (probably so since private and CV are large datasets and more correlated than public test small data).

Tom · 2026-03-03T02:59:09.660000

@cdeotte The plot I show below it's CV improvement results, about 0.6214->0.6288(+0.0074) boosts which is really close to pb boost 0.614->0.624 (+0.01). Also the public lb obtains consistent boosting: 0.588->0.596 (+0.008)

The biggest benefit and interesting thing is we choose our best surface dice model to do that, then it directly boosts topo score cv without hurting other metric, which is really insane. A classic example of how just a few lines of code can take you straight to the top.

hengck23 · 2026-03-03T03:42:20.887000

Median filer seems differentiable after some hacks. I wonder if it used as nonlinear diffusion filter in training, would it be better?

Tom · 2026-03-03T03:51:50.767000

@hengck23 When I examined the approach used in the 18th-place solution, I started thinking about the median filtering operation from a more structural perspective. A $3\times3\times3$ median filter can essentially be interpreted as repeatedly performing a local ranking over a binary mask and selecting the middle-ranked element. In this sense, the output at each voxel is primarily determined by the density (i.e., the number of active neighbors) within the $3\times3\times3$ neighborhood.

From that viewpoint, the median filter is effectively implementing a deterministic, density-based voting mechanism. This suggests that the core behavior could be approximated by a very lightweight learnable model that aggregates local neighborhood statistics and iteratively selects or refines voxel states. Such a model could mimic the “neighbor selection” dynamics of median filtering while offering greater flexibility, e.g., learning anisotropic weights, adapting to local structures, or conditioning on additional features without being constrained to a fixed rank-selection rule.

Giorgio Angelotti · 2026-03-03T08:31:28.383000

> Thank you! This is very cool. And I really like the idea that the vector field is learnt rather than being an heuristics. Did you notice a different in number of connected components after applying this stage (i.e. could it split some mergers)?

> **Tom · 2026-03-03T12:24:12.680000**
>
> @giorgioangelotti It does not explicitly guarantee merge–split correction. In practice, we handle holes and merging effects through the earlier stacked ResEncoder stages. This aspect is not currently described in the write-up. Empirically, we observe that under the same training setup, different out-of-fold (OOF) splits can lead ResUNet to either learn hole-filling behavior or sheet unmerging behavior.
>
> The primary goal of the diffeomorphic stage is to fine-tune the shape of the predicted structure. Most of the time, the overall topology is already preserved by the preceding network. However, the predicted sheet may still exhibit incorrect geometry. For example, being slightly too thick or having locally distorted shapes even if it appears visually acceptable and does not suffer from merging artifacts.
>
> At this stage, diffeomorphic refinement adjusts the geometry to better align with the ground-truth annotation. We observe a significant improvement in surface-based and topological metrics during this refinement phase.
>
> We are currently experimenting with other team's post processing since we didn't do much on that. I think diffeomorphic + good post processing is a very good combo. Like what I post above, just adding median filter repeated with 7 times, it can make huge boost on cv, lb and pb at the same time.